

Python Portfolio — Error Handling Review

Project: Python Portfolio
Technology: Python, FastAPI, SQLAlchemy, SQLite, Pydantic, JWT
Location: /home/dominic/Documents/python-portfolio
Report #: 14/20

Aspect	Status
Approach	Standard FastAPI HTTPException
Global handler	Not present
Consistency	May differ across 5 modules
Logging middleware	Not present

Error Handling Review

Error Handling Strategy

- FastAPI exception handlers for HTTP errors
- Pydantic validation for input errors
- Try/except blocks around external API calls
- Graceful degradation on service failure

Exception Hierarchy

```
HTTPException (FastAPI)
├─ 400 – Bad Request / Validation Error
├─ 401 – Unauthorized (missing/invalid JWT)
├─ 403 – Forbidden (not resource owner)
├─ 404 – Not Found
├─ 409 – Conflict (duplicate)
├─ 422 – Unprocessable Entity (Pydantic)
├─ 429 – Too Many Requests (rate limit)
└─ 500 – Internal Server Error
```

Validation Patterns

```
from pydantic import BaseModel, EmailStr, Field

class CreateURLRequest(BaseModel):
    original_url: str = Field(..., min_length=5, max_length=2048)
    custom_alias: str | None = Field(None, min_length=3, max_length=32)
```

Error Response Format

```
{
  "detail": "Human-readable error message"
}
```

Logging

Level	Usage	Example
ERROR	Unexpected failures	Database connection lost
WARNING	Rate limit exceeded	IP {ip} exceeded limit
INFO	Successful operations	User {id} logged in
DEBUG	Development only	SQL queries

Error Recovery

- Database errors: retry with backoff (not implemented)
- External API errors: return 502 Bad Gateway
- Rate limit: client should retry after Retry-After header
- File upload errors: return specific error about file format

Error Handling Gaps

- No global exception handler for unexpected errors
- External API errors not always caught gracefully
- Database constraint violations not always caught
- Some endpoints return 500 instead of specific errors